Project LUMEN

Logical User Monitoring & Explainable Network Defense

A Privacy-First, Explainable AI Approach to Phishing Detection

Author	Arkadyuti Bhattacharjee
Roll No.	2025MS006
Department	Engineering Sciences
Institution	IIITM Gwalior
Context	Smart India Hackathon 2026 (Software)
Report date	26 October 2026

1. Abstract

Phishing now reaches users through email, SMS and workplace chat, yet most defences still depend on central servers that read message content, which trades privacy for security. LUMEN is a phishing detection prototype that never uses traditional text classification on the email body. Instead, a small local language model (Ollama, *smollm2:360m*) acts as a frozen feature extractor that scores six social-engineering signals, and a GraphSAGE graph neural network classifies each email as safe or phishing from a small graph built from header, domain and URL metadata combined with those scores. The prototype is trained and evaluated on the SpamAssassin Public Corpus, and an ablation compares a keyword baseline, a metadata-only GNN and the full LLM + GNN hybrid.

2. Motivation and Problem Statement

Four gaps motivate the project:

- **Privacy versus security.** Server-side scanning exposes sensitive message content.
- **Alert fatigue.** Generic warnings do not tell users *why* a message is dangerous, so they get ignored.
- **Siloed defences.** Email, SMS and chat are usually protected by separate tools.
- **Weak keyword filters.** Fixed keyword lists are easy to evade and produce false alarms on legitimate urgent mail.

LUMEN targets these gaps by keeping inference local, using interpretable semantic scores, and combining content-derived and structural signals in a single graph model.

3. System Overview

The prototype follows a three-stage pipeline. The LLM is deliberately **not** the classifier: it is frozen, run at temperature 0, and only outputs feature scores. GraphSAGE is the only trained model.

Stage	Input	Output
1. Deterministic parsing	Headers, body URLs	Reply-To mismatch, keyword urgency, domain and URL flags
2. Local LLM extraction	Subject, From, Reply-To, body (max 5000 chars)	Six scores in [0, 1]
3. Graph classification	Per-email graph	Safe / Phishing

3.1 Deterministic features

- **reply_to_mismatch**: 1 if the Reply-To address differs from From.
- **keyword_urgency**: 1 if the email contains terms such as "urgent", "verify" or "action required" (a transparent 15-term list, kept so the old keyword approach can be compared with the LLM).
- **normalized_domain_length**: min(len(domain) / 50, 1).
- **is_ip_domain**, **is_ip_url**: whether a domain or URL uses a raw IP address.
- **uses_shortener**: whether a URL uses a known shortener such as bit.ly or tinyurl.com.

3.2 LLM semantic features

The local model receives a compact view of the email and returns only JSON with six scores, each clamped to [0, 1]:

Score	Meaning
urgency	Pressure to act quickly
credential_request	Requests for passwords, OTPs, logins or account verification
threat	Threats of suspension, deletion, legal action or loss
impersonation	Pretending to be a bank, company or administrator
financial_request	Payments, transfers, gift cards, invoices
suspicious_wording	General social-engineering language

Output is requested with a JSON schema. A tolerant parser handles JSON wrapped in markdown or surrounded by extra text, and malformed output falls back to neutral (0.0) scores and is logged. Results are cached by a stable hash of each email in *llm_features.json*, so repeat runs make no new LLM calls.

3.3 Email graph

Every email becomes one graph. Node 0 is the Email node (8 features: 2 deterministic + 6 LLM), node 1 is the Sender Domain (2 features), and each URL is its own node (2 features). Edges connect Email and Domain and Email and each URL, and are made undirected for message passing.

3.4 Model

The classifier is a node-type-aware GraphSAGE. Each node type has its own linear projection into a 64-dimensional space, plus a learned node-type embedding, so the network knows whether a node is an Email, Domain or URL without relying on zero padding. Two *SAGEConv* layers with ReLU are followed by *global_mean_pool* and a linear layer with two outputs (Safe = 0, Phishing = 1).

4. Experimental Setup

Item	Setting
Dataset	SpamAssassin Public Corpus: 20050311_spam_2 (spam) and 20030228_easy_ham_2 (ham), downloaded and parsed automatically
Split	80/20 stratified train/test, seed 42
Loss / optimiser	Cross-entropy, Adam, learning rate 0.001
Training	30 epochs, batch size 32, hidden size 64, 2 SAGE layers
Local LLM	Ollama, smollm2:360m (default); smollm2:1.7b also tried; temperature 0, context 8192
Privacy rule	No TF-IDF, word embeddings, bag-of-words or text classifiers; the body only reaches the local LLM

Three configurations are compared on the same held-out test set: **(A)** keyword baseline, which predicts phishing when *keyword_urgency* = 1; **(B)** GNN-only, using deterministic features; and **(C)** the LLM + GNN hybrid. Only training graphs are used for optimisation.

5. Results

The saved hybrid checkpoints record the following held-out test accuracy:

Hybrid model (LLM + GNN)	LLM feature extractor	Test accuracy
lumen_hybrid_graphsage.pt	smollm2:360m	79.1%
lumen_hybrid_graphsage_smollm2_1_7b.pt	smollm2:1.7b	76.6%

In this prototype the smaller 360M model scored higher than the 1.7B model (79.1% vs 76.6%). This was measured on a single split and seed, so the gap should not be read as a general ranking of the two models.

Not yet filled in: keyword-baseline accuracy, GNN-only accuracy, the hybrid-minus-GNN-only improvement, and precision / recall / F1. *train.py* prints all of these and saves them to *data/cache/results.json*; add them here before submission. Until the GNN-only number is available, this report does not claim that the LLM features improve accuracy.

6. Limitations

- The corpus is dated (2003 and 2005 mail), and spam differs from targeted phishing, so results may not transfer to current attacks.
- Results come from one split and one seed; no confidence intervals or repeated runs are reported.
- Small LLMs can be inconsistent, and failed or neutral outputs reduce feature quality.
- The graph is shallow: one email, one domain and its URLs. It does not yet model sender history or relationships across messages.
- The prototype covers email only; SMS and chat are not implemented.

7. Planned Extensions

The following parts of the full LUMEN design are **not** implemented in the current code and are listed as future work:

- **Explainable alerts:** SHAP attribution turned into plain-English warnings.
- **Active defence:** disabling malicious links in the page and injecting honey-token credentials to trap attackers.
- **Federated-lite learning:** server-side pretraining on PhishTank and OpenPhish, local fine-tuning, Gaussian differential-privacy noise on weight deltas, and cohort FedAvg aggregation.
- **Edge deployment:** a React / TypeScript browser extension and mobile app running models through TensorFlow.js or ONNX Runtime Web, extended to SMS and enterprise chat.

8. Reproducing the Work

Install the dependencies with *pip install -r requirements.txt*, pull the model with *ollama pull smollm2:360m* and keep *ollama serve* running. Then run *python train.py* to build features, train both models and print the comparison, and *python try.py* to inspect held-out predictions, error cases and a graph visualisation. A different model can be selected with the *LUMEN_OLLAMA_MODEL* environment variable.

9. Conclusion

LUMEN shows that a frozen local LLM can supply interpretable semantic features to a lightweight graph classifier without sending email content off the device. The hybrid reached 79.1% held-out accuracy with the smollm2:360m extractor. The ablation in *train.py* is what will show how much the LLM features add over metadata alone, and the privacy, explainability and active-defence components remain the next stage of the project.

References

Dwork, C., & Roth, A. (2014). The Algorithmic Foundations of Differential Privacy. *Foundations and Trends in Theoretical Computer Science*.

McMahan, H. B., et al. (2017). Communication-Efficient Learning of Deep Networks from Decentralized Data. *AISTATS*.

Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *NeurIPS*.

Wu, Z., et al. (2020). A Comprehensive Survey on Graph Neural Networks. *IEEE Transactions on Neural Networks and Learning Systems*.

Hamilton, W., Ying, Z., & Leskovec, J. (2017). Inductive Representation Learning on Large Graphs. *NeurIPS*.

SpamAssassin Public Corpus, Apache SpamAssassin Project.